Oral exam handbook

Group 15 • Final assessment preparation • 3 November 2026

Every member should be able to explain the full event path, the reason for the architecture, the measured performance, and the principal limitations. Use the short answers below as reasoning models, not scripts to recite. Follow an answer with the mechanism, an observation or test, and its scope.

The rubric allows questions to be redirected. A useful answer is technically correct and specific enough that another member can continue it without changing the claim.

The shared project explanation

> *We built a temperature observability service. It consumes experiment lifecycle and sensor events from Kafka, waits for all allocated sensors in each measurement, stores the average, exposes carry-out history through REST, and sends stabilization or out-of-range notifications. A scheduler dispatches ordered partition batches to stateless executors. PostgreSQL atomically stores state, history, notification jobs, and progress; Redis caches reads. In our final local instructor-tool run, 100 experiments with 10 sensors at 1 Hz produced 610 required notifications, all received within 5.641 seconds, with correct histories and 97.310 successful HTTP requests per second. We also identified limits under heavier combined load.*

Say “historical local run” when discussing that result. After the November cleanup, the service is stopped until recovered.

Requirements and business meaning

1 What problem are we solving

We preserve the experiment's temperature history and notify the researcher at the required transitions. We do not control heaters or cooling equipment. The application supports multiple experiments with different sensors, ranges, phases, and durations.

Follow-up: The business risk is an experiment continuing outside its required conditions without timely awareness. History supports later inspection; notification supports a timely response.

2 How do we define the temperature of the room

For one measurement ID, we take the arithmetic mean of every sensor allocated to that experiment. We wait for the complete group and require readings to agree on timestamp and hash.

Follow-up: An individual sensor can be outside the range while the room mean is inside. Replacing the mean with a maximum, median, or anomaly model would change the required behavior.

3 Why not combine the latest reading of each sensor

Those readings may represent different sampling instants. The measurement ID is the grouping identity; timestamp and hash agreement provide additional consistency checks. A correct average must refer to one complete measurement.

Show: PartialMeasurement and ExperimentStateManager.apply in models and engine.

4 What happens before the experiment starts

Configuration is persisted and can be queried immediately. During stabilization we aggregate measurements and send Stabilized on entry into range, including re-entry as the course producer does. Only the researcher's explicit start event begins carry-out.

Follow-up: Stabilization aggregates are stored internally for deduplication but excluded from the two required history endpoints.

5 Do we notify for every outside sample

No. We send OutOfRange at the first outside sample of each continuous excursion. A subsequent in-range sample enables the next excursion notification. Outside history still returns every outside carry-out sample.

Follow-up: The distinction prevents repeated alerts while preserving complete evidence of how long the experiment was outside.

6 Are the thresholds and query bounds inclusive

Yes. Equality with a temperature threshold is in range. A history sample exactly at the start or end timestamp is included. A query with equal start and end can return the sample at that instant.

Show: Strict less-than/greater-than comparisons in the engine and greater-than-or-equal/less-than-or-equal predicates in Data.

7 What do the extra REST endpoints add

POST /experiments supports explicit configuration. GET /experiments/{id} shows configuration and state. The interval endpoint groups continuous excursions and reports their start/end, with null for an ongoing excursion.

Follow-up: These additions do not change the required arrays returned by /temperature and /temperature/out-of-range.

8 Why is this not a machine-learning project

The required calculation and notification rules are deterministic and fully specified. An ML model would introduce uncertainty and validation costs without solving the stated problem. Data-science work here is experimental design, workload characterization, statistical interpretation, and trustworthy measurement.

Architecture and technology decisions

9 Why these service boundaries

Consumption/order, computation, durable data, public reads, and external delivery have different failure and load behavior. Separating them prevents a slow notification endpoint from directly blocking event computation and allows independent execution capacity.

Follow-up: Boundaries add network/serialization overhead. On one VM they do not provide host-level fault isolation, and a modular monolith would be a reasonable lower-overhead alternative.

10 Why did we choose Python

The workload is largely asynchronous I/O, with Kafka/Avro/database work supported by existing libraries. The implementation is understandable and testable, and the chosen local workload passed. We support the choice with end-to-end measurement rather than language-speed rankings.

Follow-up: If profiling shows CPU-bound state computation dominates, a faster implementation could help. It will not fix a database bottleneck by itself.

11 What exactly is stateless about an executor

It does not own authoritative experiment state across tasks. A scheduled task is a bounded batch of events; an experiment spans many tasks. The executor loads a snapshot and proposes a transaction. Its local connections and active-task counter are ephemeral; incomplete groups and range state are persisted in Data.

Follow-up: Stateless does not mean no memory or no concurrency state. It means another worker can continue from durable state.

12 Why PostgreSQL instead of only Redis

We need atomic updates across state, history, notification obligations, and checkpoints, plus durable history and uniqueness constraints. PostgreSQL supplies those directly. Redis accelerates repeat reads but is not the authority.

Follow-up: Losing Redis should reduce speed. Losing the database or its volume is a materially different failure.

13 Why retain both JSONB state and relational columns

JSONB conveniently stores configuration, phase state, and variable-sized incomplete groups. Relational measurement columns support indexed timestamp/outside queries and primary-key deduplication. The representation follows the access patterns.

Follow-up: Large partial state and frequent full JSON updates have a cost; missing sensors and unbounded growth cannot be ignored.

14 Why use Caddy if there is an api.py

api.py defines the public contract and OpenAPI and supports a Python entry point. The deployed gateway is Caddy, which forwards directly to Data or an executor and serves the generated documentation with less Python proxy work.

Show: Docker stages and Caddy routes. Internal commit/claim paths are not public routes.

15 Why least-busy dispatch

Task costs are broadly comparable, so active work plus immediate local reservations is a simple useful proxy. It avoids repeatedly choosing a replica using the same stale poll. The executor independently rejects requests when at capacity.

Follow-up: Task sizes can differ. This is not optimal cost-aware scheduling, and stale observations still require retries.

16 Why one executor if the architecture is scalable

One executor with capacity eight was the smallest measured passing configuration. Two and four replicas worsened HTTP throughput in the earlier shared-host test, despite improving some processing behavior.

Follow-up: Scalability is an ability to add capacity at the limiting stage when resources permit. It is not a rule that every replica increase must help every metric.

17 Why not directly add multiple Data replicas

The current Data process owns a local metadata-version cache and its invalidation fences. Independent replicas could serve different cached versions after one writes. Shared invalidation or removing that optimization is necessary before safe replication.

Follow-up: PostgreSQL can accept multiple clients; that alone does not make the application's caches coherent.

18 What are the autoscaling thresholds

The policy considers more than 1,024 queued events or at least 75% occupied capacity sustained demand. It can add a replica after five seconds and remove one after sixty seconds fully idle, within configurable bounds.

Follow-up: The host-side autoscaler is not automatically active in the normal Compose stack. Its load signal is not proof that the executor is the bottleneck.

Consistency and failure semantics

19 What is the most important transaction

The Data commit writes experiment versions/state, measurement rows, outbox jobs, rejected events, and the partition checkpoint together. Either they all commit or a conflict/error rolls them back.

Follow-up: The database transaction does not include the external notification HTTP request. That is precisely why the outbox exists.

20 Why store an application checkpoint when Kafka has offsets

Kafka's offset commit and PostgreSQL's transaction are separate operations. If Data commits and the Kafka acknowledgement is lost, replay must recognize that the effects already exist. The database checkpoint provides that recognition.

Follow-up: Data stores the last processed offset. Kafka is committed to the next offset to read. After processing offset 45, those values are 45 and 46.

21 What if we commit the Kafka offset first

A crash before the database transaction could leave Kafka progress ahead of the stored effects. Recovery might skip events whose history and notification job were never saved.

Show: dispatch_partition only commits after the executor's successful durable response.

22 What prevents two workers from overwriting the same state

Each proposed state update carries the version read from the snapshot. The update succeeds only if that version still matches. Checkpoint ownership is also serialized with a database row lock. A conflict rolls back and triggers reload/recomputation.

Follow-up: This is optimistic concurrency around application writes; it is not a claim that every database transaction uses SERIALIZABLE isolation.

23 How did we batch writes without losing version checks

We sorted IDs, grouped adjacent insert/update operations, and used fetchmany. Each successful operation returns a row. If returned-row count is below operation count, a version or insertion conflict occurred and the transaction aborts.

Follow-up: Grouping all inserts separately without considering lock order could undermine the original ordering discipline. The actual code groups adjacent operations after sorting.

24 What happens if a sensor message is duplicated

An identical repeated reading for an incomplete group does not add another sensor. A conflicting duplicate is invalid. A completed experiment/measurement identity is recognized on replay, including replay at new Kafka offsets.

Follow-up: For a fully completed identity, the implementation skips repeated data rather than auditing every replayed value against the original raw readings.

25 What if a sensor never arrives

The incomplete group remains in durable state. A later complete group waits behind it in timestamp order. There is a configured bound on pending groups, and a phase transition with incomplete groups is rejected.

Follow-up: The system does not fabricate a missing value. Eventual progress requires complete/correct input or an explicit future missing-sensor policy.

26 Does quarantine solve invalid input

It prevents a bad event from indefinitely blocking the Kafka partition and preserves a reason for investigation. It does not restore a missing reading, repair an invalid configuration, or make the experiment complete.

Show: Rejected-event storage/metrics and the poison-event progress test.

27 Does a Redis cache make reads eventually consistent

A cache does not inherently determine that. Here the key contains a state version, and a single Data writer invalidates metadata before and after writes while fencing concurrent fills. Reads overlapping a transaction can observe a valid earlier snapshot; reads after acknowledged writes should select the new version.

Follow-up: That correctness argument depends on the single-process invalidation ownership.

28 Why is external notification delivery at least once

The receiver may accept a request just before the sender fails to record success. When the lease expires, retry can deliver it again. The job identity is deduplicated in our database, but the receiver must honor idempotency to prevent repeated acceptance.

Avoid: “Kafka gives us exactly-once notifications.”

29 What do notification leases accomplish

They limit simultaneous ownership of due jobs and permit recovery when a worker disappears. Claims skip rows locked by another claim. A result must carry the current lease token, so an old worker cannot acknowledge a job after ownership changes.

Follow-up: A lease does not force an already-sent HTTP request to stop. A very delayed sender can still overlap an external retry.

30 Are retries always good for the ten-second target

Retries improve eventual delivery but consume time. The code retries transport errors and selected statuses with bounded delay, and records permanent request failures. A fifteen-second lease after a sender crash can already exceed the deadline.

Follow-up: Durable recovery and bounded steady-state latency are different guarantees. The current system does not promise the deadline through arbitrary failures.

31 What happens if PostgreSQL is unavailable

Data cannot durably accept the task. The executor/scheduler retry, and Kafka offsets are not acknowledged for uncommitted work. The queue can fill and intake can pause. Correctness is favored over pretending the write succeeded, but availability and latency degrade.

32 What does a consistent snapshot mean in our code

The load endpoint reads checkpoint, versioned states, and completed IDs in one repeatable-read transaction. A history cache miss obtains metadata and points in one SQL statement. Those reads avoid combining pieces from different committed moments.

Follow-up: A snapshot can be consistent without being the latest possible state at every instant during concurrent writes.

Performance and experimental reasoning

33 What is the most convincing result

The final local run of the original instructor tools: 100 concurrent experiments, ten sensors each, 1 Hz, sixty carry-out samples, and 100 offered HTTP/s. All 610 required notifications arrived, maximum 5.641 seconds; all 202 complete history comparisons were correct; achieved HTTP rate was 97.310/s.

Follow-up: One warmup ended before the main concurrency. This is historical local evidence, not a blanket hosted-grading pass.

34 What failed before that result

The initial instructor stress run received all 632 notifications and had correct histories, but three notifications exceeded ten seconds, maximum 10.716 seconds. Achieved HTTP throughput was 86.848/s. Per-experiment state-write round trips were the concrete optimization target.

35 Why did the expected notification count change

The simulator produces random temperature trajectories, and notifications depend on transitions. Same configured workload does not imply the same number of excursions. The final run required 610, and all 610 were received.

Follow-up: This is why we avoid presenting a single before/after pair as a precisely isolated causal experiment.

36 Is the performance gain statistically proven

No repeated randomized A/B series is preserved. The observed maximum fell 47.36% and HTTP rate rose 12.05%, but randomness, resource state, and workload history can contribute. The mechanism is plausible, the full tests passed, and repeat runs would strengthen attribution.

37 What does keeping up with HTTP mean

Valid completions must track the offered workload over the same time window without steadily growing queues or hidden drain time. Error rate and latency also matter. Our local 95% rule is explicit engineering acceptance, not a percentage supplied by the professor.

Follow-up: A generator's "effective rate" is not automatically the server's validated completion rate.

38 Why use independently paced requests

If a client waits for every response before scheduling another request, overload can reduce offered load and conceal a growing service-time problem. Our deterministic harness schedules independently and counts client-queue time and drain time.

Follow-up: The load generator itself can saturate on the same host, so its offered/achieved rates must be observed too.

39 Is p99 below ten seconds sufficient

No. At least one notification can exceed the deadline while p99 remains below it, and a missing notification may have no latency sample at all. We need required/received counts and a strict maximum below ten seconds.

Follow-up: The current harness accepts the exactly-ten-second boundary; the rubric says below ten. The historical passing maximum is well below both.

40 Why did more executors hurt

The added execution concurrency competed for the same CPU and downstream Data/database capacity. Earlier matrix results showed achieved HTTP around 93.4–93.6/s with two/four executors versus 98.077/s with one.

Follow-up: We observed the effect; separating CPU from database contention precisely needs further timing/profiling. Do not pretend the matrix alone measured every cause.

41 How would you pinpoint a bottleneck

Compare demand and output at each stage. Queue growth with high CPU suggests compute contention; queue/occupancy with database waits suggests Data writes; slow HTTP with healthy ingestion suggests history reads; current history with an aging outbox suggests delivery. Confirm with a targeted experiment.

Follow-up: More replicas are a hypothesis-driven intervention, not the first answer to every latency problem.

42 What is the theoretical concurrency limit

With sixteen partitions and executor capacity eight, at most min(16, 8×replicas) partition tasks can be active under this scheduler design. Throughput also depends on service time, batch size, skew, and shared resources.

Follow-up: Four executors advertising 32 slots do not create 32 independent partitions.

43 How do sensor count and experiment duration change the workload

At E experiments, S sensors, and T seconds/sample, input rate is E×S/T and aggregate rate is approximately E/T. Duration grows retained history and query size. At 100 experiments and 1 Hz, continuous carry-out yields 8.64 million aggregate points per day.

Follow-up: “Only 100 experiments” does not imply bounded storage or arbitrary sensor capacity.

44 What should we test next

Restore and reproduce the baseline, retest old failing fixed profiles on the final code, then measure continuous changing HTTP/event rates, burst recovery, skew, cache warmth, and longer history/soak behavior. Record all runs, including failures, and change one factor at a time when diagnosing causes.

Operations, costs, and project ownership

45 What did the Docker cleanup remove

It killed and removed containers and removed image references; those application processes and their writable container layers are gone. Named volumes remain, but that alone does not verify database contents or recovery. No containers were present in the November audit.

Follow-up: Docker describing an unused volume as reclaimable does not mean it is disposable. Restore and validate before saying the application is healthy.

46 What do we listen to during the professor's demo

The shared experiment topic, using the demo overlay and group15-temperature-demo-v1. Notifications go to the external course endpoint with the supplied token. Local instructor tests use course-test-events and a local notification validator.

Follow-up: Source configuration shows intended mode. Actual partition assignment and fresh processing show that it is running in that mode.

47 What was wrong with external Kafka TLS

The public hostname did not match the broker certificate names. The demo-specific setting disabled hostname matching while retaining CA validation and client-certificate authentication. Strict matching remains the general default.

Follow-up: Encryption and CA trust do not replace hostname verification. The preferred external fix is a certificate valid for the public name, after which strict mode should be restored.

48 Which parts of the dashboard are ours

The port-3004 operational dashboard is ours. The port-3009 grading view derives from the original instructor dashboard with local bindings. The bottom Latest local test panel, ID 1000, was added by us and reflects explicit runner checks. The hosted Grafana belongs to the course.

Follow-up: No-data green panels are not evidence. A pinned historical time window is useful, but it must not be confused with current live health.

49 How do we calculate costs and what is missing

Configured VM cost/hour plus measured database GB times configured storage GB-month price divided by 730. Rates default to zero with an explicit unconfigured indicator. It omits allocated unused storage, network, monitoring, and external infrastructure unless separately accounted for.

Follow-up: The full-disk incident shows why actual filesystem usage and recoverability matter even when the database-only cost estimate is small.

50 What did you personally contribute

Give a truthful concrete example with an artifact: code, test, operational recovery, benchmark analysis, documentation, or a design investigation. Explain the decision and how you checked it. Do not invent a division of labor based on degree labels or claim ownership of work you cannot explain.

Follow-up: Every member should still explain the rest of the architecture because the rubric assesses individual knowledge across the project.

Worked exercise 1 Sensor means and excursions

Use three sensors and range 25–26 inclusive. All rows represent complete groups with matching timestamp/hash.

Phase and sample	Sensor temperatures	Mean	Notification	Appears in required history
Stabilizing S1	20, 21, 22	21	None	No
Stabilizing S2	24, 25, 26	25	Stabilized	No
Running M1	24, 26, 28	26	None	Yes
Running M2	26, 27, 28	27	OutOfRange	Yes
Running M3	27, 28, 29	28	None	Yes
Running M4	24, 25, 26	25	None	Yes
Running M5	22, 23, 24	23	OutOfRange	Yes

There are five carry-out history points, three outside-history points (M2, M3, M5), two carry-out notifications, and one stabilization notification. The first excursion closes at M4. If M5 remains outside until termination, its interval closes at termination.

Now remove sensor C from M2. No M2 mean or notification can be emitted. M3, even if complete, waits behind the earlier partial group. This is a useful example of correctness requiring input completeness.

Worked exercise 2 Transaction and replay

Start with database checkpoint 40. A task processes offsets 41 through 45 and creates an average and outbox job. Data commits checkpoint 45, but the executor's response is lost before the scheduler acknowledges Kafka.

Kafka may resend 41–45. The reloaded checkpoint shows they are already durable, so no second average or job is needed. Kafka can then commit next-read offset 46. If the original database transaction had failed before commit, the checkpoint would still be 40 and the events would be recomputed.

Now change the failure: the notification receiver accepted the job, but the sender lost its acknowledgement. Database checkpointing does not prove whether that external acceptance happened. Retry may duplicate it unless the receiver honors the idempotency identity.

Worked exercise 3 Bottleneck diagnosis

Observe three windows:

Window	Observation	First hypothesis and next check
A	CPU near saturation, scheduler queue rises, HTTP p95 rises after extra replicas	Shared-host contention; compare process CPU and reduce executor competition at the same offered load
B	Histories current, outbox oldest age rises, many delivery 503s	Receiver or network trouble; inspect delivery outcomes/timeouts rather than adding executors
C	Free executor slots, one partition far behind the others	Key skew or sequential partition work; inspect per-partition progress before adding workers

A strong answer says what additional observation would falsify the hypothesis. It does not diagnose only from one correlated graph.

Source navigation drill

Before the exam, each member should locate and explain these without searching the whole repository blindly:

File	Explain aloud
engine.py	Complete grouping and previous_out_of_range
executor.py	prepare_commit, replay shortcut, conflict loop
scheduler.py	dispatch_partition, offset commit, pause/resume
data.py	Atomic commit, fetchmany/version check, metadata fence, claim/result
notification.py	Lease acknowledgement and external duplicate window
schema.sql	Uniqueness and persistence boundaries
compose.demo.yml	Topic, consumer group, credentials, notification destination
loadtests/course.py	Real-load pass conditions and the added panel
tests/test_data_integration.py	Stale-middle-version rollback
benchmarks/2026-10-09/course-stress.json	Workload and measured result

The architecture report explains the mechanisms; the performance report links each numerical claim to its source.